

Introduction to Queue Data Structure

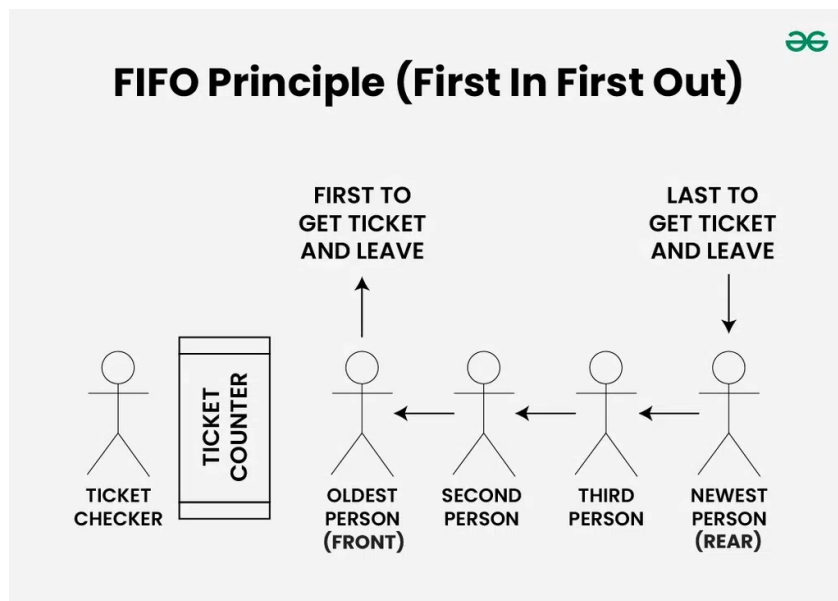
Last Updated : 28 Mar, 2025



Queue is a linear data structure that follows **FIFO (First In First Out) Principle**, so the first element inserted is the first to be popped out.

FIFO Principle in Queue:

FIFO Principle states that the first element added to the Queue will be the first one to be removed or processed. So, Queue is like a line of people waiting to purchase tickets, where the first person in line is the first person served. (i.e. First Come First Serve).

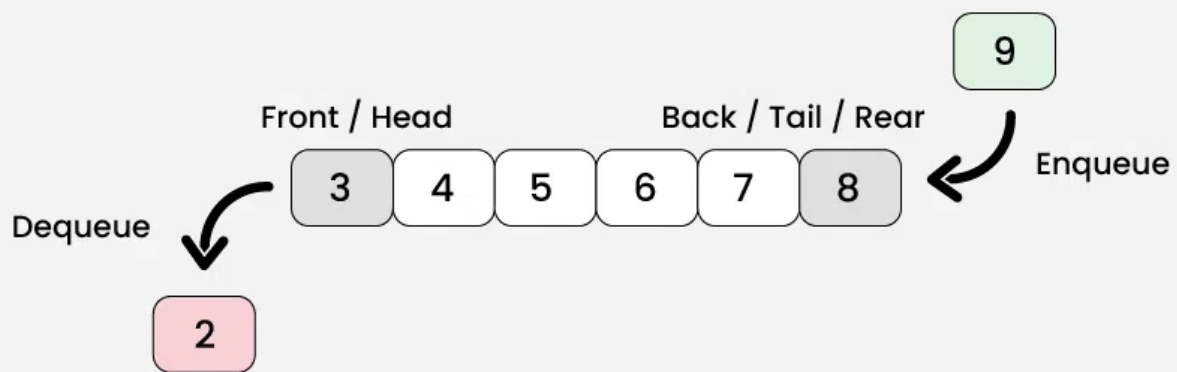


Basic Terminologies of Queue

- **Front:** Position of the entry in a queue ready to be served, that is, the first entry that will be removed from the queue, is called the **front** of the queue. It is also referred as the **head** of the queue.
- **Rear:** Position of the last entry in the queue, that is, the one most recently added, is called the **rear** of the queue. It is also referred as the **tail** of the queue.
- **Size:** Size refers to the **current** number of elements in the queue.
- **Capacity:** Capacity refers to the **maximum** number of elements the queue can hold.

Representation of Queue

Representation of Queue Data Structure



Queue Operations

1. **Enqueue:** Adds an element to the end (rear) of the queue. If the queue is full, an overflow error occurs.
2. **Dequeue:** Removes the element from the front of the queue. If the queue is empty, an underflow error occurs.
3. **Peek/Front:** Returns the element at the front without removing it.
4. **Size:** Returns the number of elements in the queue.
5. **isEmpty:** Returns true if the queue is empty, otherwise false.
6. **isFull:** Returns true if the queue is full, otherwise false.

For detailed steps and more information on each operation, Read [Basic Operations for Queue in Data Structure](#).

Implementation of Queue Data Structure

Queue can be implemented using following data structures:

- [Simple Array implementation of Queue](#)
- [Efficient Array Implementation of Queue](#)
- [Implementation of Queue using Linked List](#)

Complexity Analysis of Operations on Queue

Operations	Time Complexity	Space Complexity
enqueue	$O(1)$	$O(1)$
dequeue	$O(1)$	$O(1)$
front	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
isFull	$O(1)$	$O(1)$

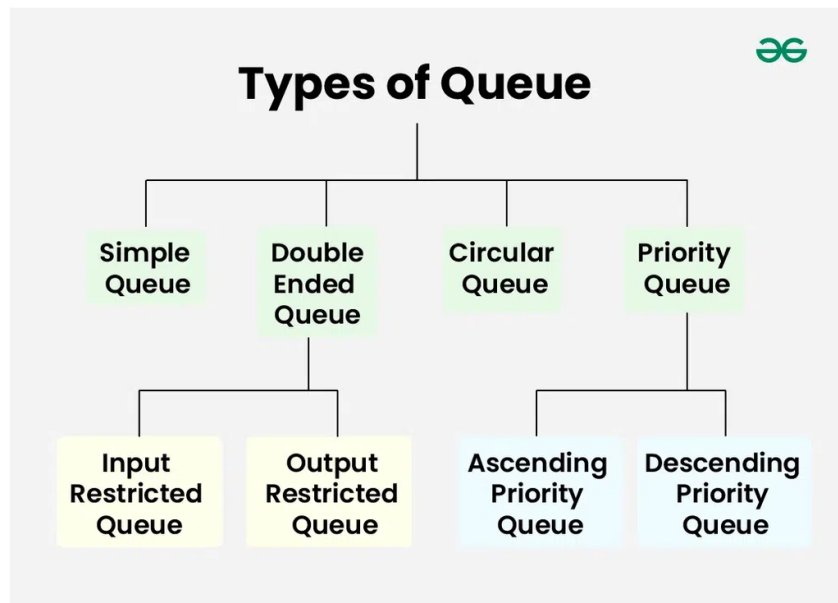
Types of Queues

Queue data structure can be classified into 4 types:

1. **Simple Queue:** Simple Queue simply follows **FIFO** Structure. We can only insert the element at the back and remove the element from the front of the queue. A simple queue is efficiently implemented either using a linked list or a circular array.
2. **Double-Ended Queue (Deque):** In a [double-ended queue](#) the insertion and deletion operations, both can be performed from both ends. They are of two types:
 - **Input Restricted Queue:** This is a simple queue. In this type of queue, the input can be taken from only one end but deletion can be done from any of the ends.
 - **Output Restricted Queue:** This is also a simple queue. In this type of queue, the input can be taken from both ends but deletion can be done from only one end.

3. **Priority Queue:** A [priority queue](#) is a special queue where the elements are accessed based on the priority assigned to them. They are of two types:

- **Ascending Priority Queue:** In Ascending Priority Queue, the elements are arranged in increasing order of their priority values. Element with smallest priority value is popped first.
- **Descending Priority Queue:** In Descending Priority Queue, the elements are arranged in decreasing order of their priority values. Element with largest priority is popped first.



types of queues

Applications of Queue Data Structure

Application of queue is common. In a computer system, there may be queues of tasks waiting for the printer, for access to disk storage, or even in a time-sharing system, for use of the CPU. Within a single program, there may be multiple requests to be kept in a queue, or one task may create other tasks, which must be done in turn by keeping them in a queue.

- A Queue is always used as a buffer when we have a speed mismatch between a producer and consumer. For example keyboard and CPU.
- Queue can be used where we have a single resource and multiple consumers like a single CPU and multiple processes.
- In a network, a queue is used in devices such as a router/switch and mail queue.

- Queue can be used in various algorithm techniques like Breadth First Search, Topological Sort, etc.

Please refer [Applications of Queue](#) for more details.

- [Queue Operations](#)
- [Applications, Advantages and Disadvantages of Queue](#)
- [Maximum of all subarrays of size k](#)
- [Generate Binary Numbers](#)
- [Queue using two Stacks](#)
- [Reverse First K elements of Queue](#)

Queue Data Structure

[Visit Course ↗](#)[Comment](#)[More info ▼](#)[Advertise with us](#)[Next Article >](#)

Introduction and Array Implementation
of Queue

Similar Reads

Queue Data Structure

A Queue Data Structure is a fundamental concept in computer science used for storing and managing data in a specific order. It follows the principle of "First in, First out" (FIFO), where the first element added...

🕒 2 min read

Introduction to Queue Data Structure

Queue is a linear data structure that follows FIFO (First In First Out) Principle, so the first element inserted is the first to be popped out. FIFO Principle in Queue: FIFO Principle states that the first element...

🕒 5 min read

Introduction and Array Implementation of Queue

Similar to Stack, Queue is a linear data structure that follows a particular order in which the operations are performed for storing data. The order is First In First Out (FIFO). One can imagine a queue as a line o...

🕒 2 min read

Queue - Linked List Implementation

In this article, the Linked List implementation of the queue data structure is discussed and implemented. Print '-1' if the queue is empty. Approach: To solve the problem follow the below idea: we maintain two...

🕒 8 min read

Applications, Advantages and Disadvantages of Queue

A Queue is a linear data structure. This data structure follows a particular order in which the operations are performed. The order is First In First Out (FIFO). It means that the element that is inserted first in the...

🕒 5 min read

Different Types of Queues and its Applications

Introduction : A Queue is a linear structure that follows a particular order in which the operations are performed. The order is First In First Out (FIFO). A good example of a queue is any queue of consumers...

🕒 8 min read

Queue implementation in different languages



Some question related to Queue implementation



Easy problems on Queue



Intermediate problems on Queue

